

MESIA: Understanding and Leveraging Supplementary Nature of Method-level Comments for Automatic Comment Generation¹

Xinglu Pan²
SCS Peking University; Key Lab of
HCST (PKU), MOE
Beijing, China
bjdpxpl@pku.edu.cn

Chenxiao Liu³
SCS Peking University; Key Lab of
HCST (PKU), MOE
Beijing, China
jslxcx@pku.edu.cn

Yanzhen Zou⁴
SCS Peking University; Key Lab of
HCST (PKU), MOE
Beijing, China
zouyz@pku.edu.cn

Tao Xie⁵
SCS Peking University; Key Lab of
HCST (PKU), MOE
Beijing, China
taoxie@pku.edu.cn

Bing Xie*⁶
SCS Peking University; Key Lab of
HCST (PKU), MOE
Beijing, China
xiebing@pku.edu.cn

ABSTRACT⁷

Code comments are important for developers in program comprehension. In scenarios of comprehending and reusing a method, developers expect code comments to provide supplementary information beyond the method signature. However, the extent of such supplementary information varies a lot in different code comments. In this paper, we raise the awareness of the supplementary nature of method-level comments and propose a new metric named MESIA (**Mean Supplementary Information Amount**) to assess the extent of supplementary information that a code comment can provide. With the MESIA metric, we conduct experiments on a popular code-comment dataset and three common types of neural approaches to generate method-level comments. Our experimental results demonstrate the value of our proposed work with a number of findings. (1) Small-MESIA comments occupy around 20% of the dataset and mostly fall into only the WHAT comment category. (2) Being able to provide various kinds of essential information, large-MESIA comments in the dataset are difficult for existing neural approaches to generate. (3) We can improve the capability of existing neural approaches to generate large-MESIA comments by reducing the proportion of small-MESIA comments in the training set. (4) The retrained model can generate large-MESIA comments that convey essential meaningful supplementary information for methods in the small-MESIA test set, but will get a lower BLEU score in evaluation. These findings indicate that with good training data, auto-generated comments can sometimes even surpass human-written reference comments, and having no appropriate ground truth for evaluation is an issue that needs to be addressed by future work on automatic comment generation.

*Bing Xie is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICPC '24, April 15–16, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0586-1/24/04...\$15.00
<https://doi.org/10.1145/3643916.3644401>

CCS CONCEPTS⁹

• Software and its engineering → Documentation.¹⁰

KEYWORDS¹¹

Code comment, Comment generation, Deep learning, Evaluation¹²

ACM Reference Format:¹³

Xinglu Pan, Chenxiao Liu, Yanzhen Zou, Tao Xie, and Bing Xie. 2024. MESIA: Understanding and Leveraging Supplementary Nature of Method-level Comments for Automatic Comment Generation. In *32nd IEEE/ACM International Conference on Program Comprehension (ICPC '24)*, April 15–16, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3643916.3644401>¹⁴

1 INTRODUCTION¹⁵

As modern software continues to grow in complexity, code comments become increasingly important in program comprehension [15, 22, 29, 58, 63–66]. Code comments are specially essential for methods. In scenarios of comprehending and reusing a method, developers often access only the method signature (without access to the method body or looking into it when accessible), which includes the method's name, parameters, and return type. Although the method signature conveys valuable information for developers, it may be too short [24] to express all essential information needed to comprehend the method. As a result, developers have to turn to code comments to get supplementary (i.e., additional) information to better comprehend the method.

Developers typically follow a **hybrid mode** for reading the method-level comment to comprehend a method where developers read both the method signature and the comment. As mentioned earlier, because developers have digested the method signature, they expect the comment to provide more supplementary information beyond the method signature [27, 42]. According to a recent study [27] on professional developers' expectations on code comments, developers state that “code is the best documentation” and code comments shall provide supplementary information beyond what can be easily obtained from code [42], especially the method signature.

Although developers expect code comments to provide supplementary information, we have observed that the extent of supplementary information varies a lot in different code comments as

```

/**
 * remove a property change listener.
 */
public void removePropertyChangeListener(PropertyChangeListener pcl){...}
(a)

/**
 * marks the specified entry as used by setting its last used time to the current time in nanoseconds.
 */
protected void markUsed (Entry entry){...}
(b)

```

Figure 1: Different code comments that provide different extents of supplementary information.

shown in the following two examples¹. (1) Figure 1(a) shows a code comment that provides little supplementary information. The comment of the method “*removePropertyChangeListener*” is “*remove a property change listener*”, which provides only an “a” besides the content that can be directly obtained from the method name. (2) Compared with the preceding example, the comment of the method “*markUsed*” in Figure 1(b) can provide a larger extent of supplementary information. Here a large part of the comment content (such as “*by setting its last used time to the current time in nanoseconds*”) is supplementary information that cannot be directly obtained from the method signature.

Toward supporting the hybrid mode of reading method-level comments to comprehend a method, in this paper, we propose a new metric named MESIA (**M**ean **S**upplementary **I**nformation **A**mount) to quantify the extent of supplementary information that a method-level comment can provide beyond the information reflected by the method signature. The insight of the MESIA metric comes from the self-information given by the Shannon information theory [48]. It quantifies the information amount of a word by the surprise level of its particular outcome. In particular, to compute MESIA, we first calculate the information amount of each comment word by the possibility of the word appearing in all comments in a dataset. We then calculate the supplementary extent of a method comment by considering the information amount of all words in the comment, all words in its related method signature, and the length of the comment, respectively.

We apply the MESIA metric on a popular code-comment dataset named TL-CodeSum [1]. Our empirical results show that MESIA is consistent with manual assessment of the relative supplementary extent of the code comment beyond the method signature of a method. We investigate the distribution of various kinds of code comments in different value ranges of MESIA to understand the supplementary nature of method-level comments. The results show that small-MESIA-value (in short as small-MESIA) comments (with less than 3 MESIA value), occupying around 20% of the dataset, mostly fall into only the WHAT comment category (describing the functionality) [14] while the large-MESIA-value (in short as large-MESIA) comments (with greater than 6 MESIA value) can widely cover all six comment categories.

¹Both examples are from an existing code-comment dataset named TL-CodeSum [1] studied in this paper.

We further apply our proposed MESIA metric to three common types of neural approaches [9, 28, 61] to generate code comments with different MESIA values. By training a neural network model with pairs of a code snippet (usually in the method-level granularity) and its comment, the model can learn to translate a code snippet to its comment. Our results show that the existing neural approaches are far from satisfactory when used to generate large-MESIA comments, reflecting challenging and important cases for future research efforts to tackle. Subsequently, we explore strategies that leverage MESIA to improve the generation of large-MESIA comments. The results indicate that we can improve the capability of existing neural approaches to generate large-MESIA comments by reducing the proportion of small-MESIA comments in the training set. In addition, the retrained model can generate large-MESIA comments that convey essential meaningful supplementary information for methods in the small-MESIA test set, but will get a lower BLEU score in evaluation. These findings indicate that with good training data, auto-generated comments can sometimes even surpass human-written reference comments, and having no appropriate ground truth for evaluation is an issue that needs to be addressed by future work on automatic comment generation. Finally, we discuss the challenges of generating large-MESIA comments for future work on automatic comment generation to tackle.

Our work raises the awareness of developers’ expectations for the supplementary information of method-level comments in a hybrid mode of reading the method-level comment to comprehend a method. Our experimental results reveal the influence of the supplementary nature of method-level comments on automatic comment generation. The results can guide the usage of data in future work to effectively generate comments that support developers’ hybrid mode of comprehending a method.

In summary, this paper makes the following main contributions:

- A new metric named MESIA to assess the extent of supplementary information that a code comment for a method can provide beyond the method signature.
- An empirical investigation of the MESIA metric on a popular code-comment dataset, showing that small-MESIA comments occupy around 20% of the dataset and mostly fall into only the WHAT comment category, while large-MESIA comments can provide various kinds of essential information about methods.

- An evaluation upon three common types of neural approaches for generating method-level comments, showing that the existing approaches are far from satisfactory in generating the large-MESIA comments.
- A strategy to improve automatic generation of large-MESIA comments with existing neural approaches by reducing the proportion of small-MESIA comments in the training set.

The rest of the paper is organized as follows. Section 2 describes the motivation and challenges addressed by our work. Section 3 illustrates our proposed MESIA metric. Section 4 describes the study design of our research. Section 5 presents the study results. Section 6 discusses the implications and threats to validity. Section 7 describes the related work. Finally, Section 8 concludes this paper.

2 MOTIVATION AND CHALLENGES

In this section, we explain our motivation and challenges to be addressed by our work.

Our motivation comes from the observation that in the hybrid mode of reading the method-level comment for a method, developers expect the comment to provide more supplementary information beyond the method signature. So besides the existing widely considered aspects in assessing code comments [46] (such as accuracy [39, 52], adequacy [41], and naturalness [28]), we also need a metric to assess such supplementary extent of the code comment beyond the method signature. The assessment should concern, but not limit to, the following three factors.

First, we need to consider how to evaluate the information amount of a given code comment. The information amount of a code comment relies on the information amount of its words, which varies a lot in the same dataset. Take the comment “marks the specified entry as used by setting its last used time to the current time in nanoseconds” as an example, words such as “the”, “specified”, and “to” are often used in code comments of the dataset and they can provide little information. Words such as “setting”, “current”, and “time” are less often used and they provide more information to developers.

Second, we need to consider the supplementary information amount of the given code comment beyond the method signature. Code is the best documentation [27] and developers can directly get some information from the method signature [23], especially when it follows a good naming convention. For example, given the method signature “markUsed(Entry entry)”, developers can directly know that this method will mark the entry used. It is necessary to reduce the impact of the method signature in the metric.

Third, we need to consider the length (how many words) of the given code comment. Because code comments in different methods inherently have different lengths, the absolute supplementary information amount of the code comments can vary greatly. What we care about is, under the comment length, whether the comment has provided a relatively large extent of supplementary information. Therefore, we should consider the length of the comment to measure its supplementary extent so that the result is comparable across different comment entries.

To get an initial understanding of these three factors, we conduct a two-step analysis on an existing dataset [1]. This dataset is widely used by existing neural approaches for generating method-level

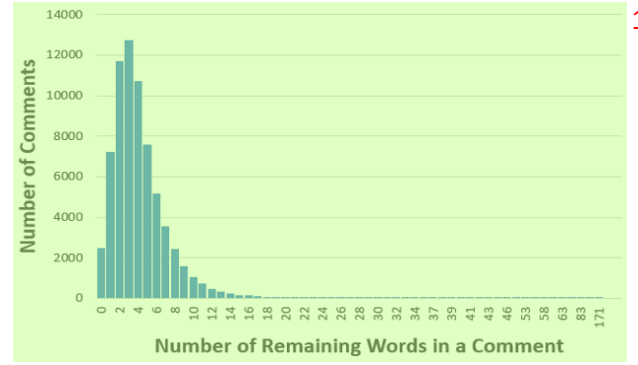


Figure 2: An analysis of the number of words that remain in a method comment after removing stop words and words in the method’s split signature.

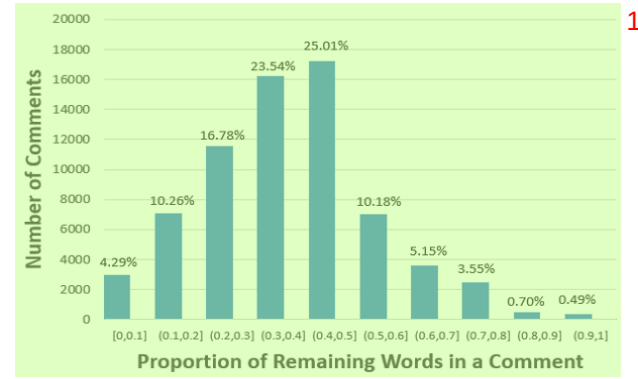


Figure 3: An analysis of the proportion of words that remain in a method comment after removing stop words and words in the method’s split signature.

comments. First, we calculate the number of words that remain in the given code comment after we remove the stop words and the words appearing in its method signature. Second, we calculate the proportion of the remaining words in the given code comment by further considering its original length. In the analysis, we split the words in the method signature. The split comprehensively considers the camel case spelling and snake case spelling. Then we stem these split words and words in the given code comment to a normalized form for comparison. For example, the comment of the method “markUsed” is “marks the specified entry as used by setting its last used time to the current time in nanoseconds.” After removing the stop words (“the”, “as”, “by”, “its”, “to”, “in”), the words in the split method name (“marks”, “used”), and the words in the split parameter’s name (“entry”), seven words (“specified”, “setting”, “last”, “time”, “current”, “time”, “nanoseconds”) remain. The proportion of the remaining words in the comment is $7/18 \approx 38.9\%$.

From Figure 2, we can see that after we remove the stop words and the words in the method’s split signature, the number of remaining words in a code comment mainly focuses on [0,8] and there is a long tail. So referring to the results of Figure 3, we need to evaluate

the extent of supplementary information for different code comments based on the proportion of their remaining words. However, the specific calculation is still a great challenge because different remaining words can provide different amount of information.

3 PRELIMINARY AND DEFINITION

As described in the preceding section, we want to evaluate the extent of supplementary information in a method comment beyond its method signature. We borrow the idea from Shannon information theory and propose a new metric named MESIA (**M**ean **S**upplementary **I**nformation **A**mount) to assess code comments.

In 1948, Shannon [48] derived a metric of information content named the self-information of a message x :

$$I(x) = -\log(p(x)) \quad (5)$$

Here $p(x)$ is the probability that message x is chosen from all possible choices in the message space X . The self-information can be interpreted as quantifying the surprise level of a particular outcome. When a message with less appearing possibility occurs, it brings more information.

To calculate the information amount of a given code comment, we first follow the self-information to measure the information amount for each word w in the given code comment as

$$I(w) = -\log(p(w)) \quad (8)$$

Here $p(w)$ represents the possibility of w appearing in code comments. The basic idea behind this equation is that if a word is more likely to appear in code comments, the less extra information it can bring to developers.

Thus we calculate the information amount for the given code comment by summing up the information amount of each word in the comment. We define a code comment in a dataset as $C = \langle w_1, w_2, \dots, w_n \rangle$. All code comments in the dataset are defined as $Comments = \{C_1, C_2, \dots, C_m\}$. All words used in comments of the dataset are defined as $W = \bigcup_{i=1}^m \{w | w \in C_i\}$. We use the frequency of each word x (referred to as $freq(x)$) in W to calculate the possibility for each word w in the given code comment.

$$p(w|W) = freq(w) / \sum_{x \in W} (freq(x)) \quad (11)$$

We then define the information amount of a comment C in the dataset as

$$I(C|W) = - \sum_{w \in C} \log(p(w|W)) \quad (13)$$

Second, we further consider the method signature in the metric. Code comments serve as a way to provide supplementary information in program comprehension. If developers can easily obtain the comment's content from the method signature, then the comment brings little supplementary information. Therefore, we define the supplementary information amount of a comment C beyond its method signature as

$$I(C|Code, W) = - \sum_{w \in C} \log(p(w|Code, W)) \quad (15)$$

Here the “Code” in the formula refers to the words in the method's split signature. We consider the method signature in the metric as presented in the following formulas

$$p(w|Code, W) = \begin{cases} 1 & w \in Code \\ p(w|W) & w \notin Code \end{cases} \quad (17)$$

When w appears in the method's split signature, we set $p(w|Code, W)$ to 1. If w does not appear in the method's split signature, we set $p(w|Code, W)$ to $p(w|W)$. We split the method signature into words according to two heuristic rules: (1) camel case spelling and (2) snake case spelling. We stem these split words and words in the given code comment to a normalized form for comparison. The details have been introduced in Section 2.

Finally, the length of the given comment must be considered because we want to evaluate the extent of supplementary information for code comments. Therefore, we define the MESIA (**M**ean **S**upplementary **I**nformation **A**mount) of a comment C as

$$\begin{aligned} MESIA(C) &= I(C|Code, W) / \text{len}(C) \\ &= - \sum_{w \in C} \log(p(w|Code, W)) / \text{len}(C) \end{aligned} \quad (18)$$

In the following sections, we use the MESIA metric to assess code comments in an existing dataset and conduct several experiments.

4 STUDY DESIGN

In this section, we present the study design for investigating the MESIA metric toward understanding the supplementary nature of method-level comments in an existing dataset and understanding the supplementary nature's influence on three common types of neural approaches to generate method-level comments. We mainly aim to answer three research questions.

4.1 Research Questions

RQ1. How well does MESIA reflect the relative extent of supplementary information in method-level comments?

Answering this research question helps investigate whether MESIA is consistent with manual assessment on the relative supplementary extent of code comments. We also want to find out the distribution of various kinds of code comments in different value ranges of MESIA to better understand the supplementary nature of method-level comments.

RQ2. What is the capability of existing neural approaches to generate code comments with different MESIA values?

In this paper, we raise the awareness of the supplementary nature of method-level comments. Answering this research question helps investigate the capability of existing neural approaches to generate code comments with various extents of supplementary information.

RQ3. How well can MESIA be used to improve existing neural approaches to generate large-MESIA comments?

The neural approaches for generating code comments require a large amount of code-comment data. The training data can substantially influence the comment-generation result. Answering this research question helps investigate whether we can help existing neural approaches to generate code comments that better support developers in a hybrid mode of reading method-level comments after we leverage MESIA to refine the training data.

Table 1: The TL-CodeSum Dataset.

	#Training Pairs	#Validation Pairs	#Test Pairs
Original	69708	8714	8714
Denoised	53597	7562	7584
Deduplicated	53506	6040	5905

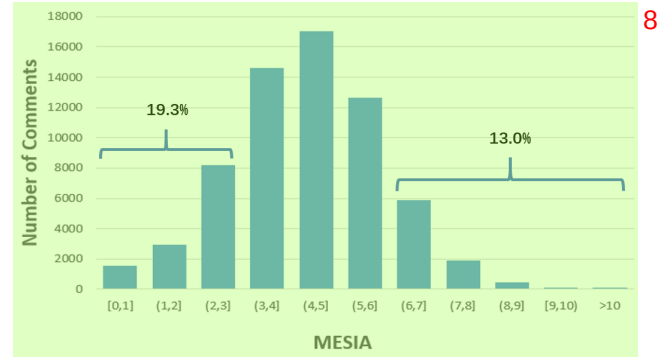
4.2 Study Setup

The background of our study is the recent increasing popularity of neural approaches [9, 11, 25, 28, 34–36, 57, 60, 62, 67] to generate method-level comments. These approaches typically consist of three steps. First, these neural approaches construct a dataset consisting of pairs of a method and its comment. The dataset is divided into three parts: training, validation, and test sets. Second, using the training and validation sets, the neural approaches train a neural network model, such as the sequence-to-sequence (seq2seq) model [56] or the Transformer model [59], to learn the alignment of a method and its comment. Third, using the neural network model, the neural approaches generate a comment for the given method in test set and evaluate the generated comment by calculating its similarity with the corresponding reference comment via metrics such as BLEU [43].

4.2.1 Dataset. We use a popular code-comment dataset named TL-CodeSum [1] provided by Hu et al. [26] to conduct our study. This dataset originally contains about 87K pairs of a method and its comment collected from more than 9K open-source Java projects created from 2015 to 2016 with at least 20 stars in GitHub. Hu et al. extract the methods and their corresponding Javadoc comments, and take the first sentence of the comments and use some heuristic rules to remove some methods such as setter, getter, and test methods.

Recently, much work [49, 50] has pointed out the quality issues of existing datasets of code comments. Specifically, Shi et al. [50] point out that noise intensively exists in existing datasets and the TL-CodeSum dataset contains a certain percentage of data duplication. The duplication of data will influence the evaluation of existing approaches because the generalization ability of the model cannot be well assessed. Therefore, we use the denoised version [5] of the TL-CodeSum dataset provided by Shi et al. [50] and further deduplicate it to conduct our study. The deduplication process is as follows: (1) We compare code-comment samples in the training set and remove duplicated samples so that each sample in the training set is unique. (2) Following (1), we deduplicate the validation set and further remove samples of the validation set that appear in the training set. (3) Following (1), we deduplicate the test set and further remove samples in the test set that appear in the validation set or the training set. Table 1 shows the detail of the dataset used in our study.

4.2.2 Neural Approaches. We use MESIA to investigate three common types of neural approaches [9, 28, 61] for generating method-level comments and study their results. The first approach uses the seq2seq model [56], which contains two layers of the LSTM encoder and one layer of the LSTM decoder. The second approach uses the Transformer model [59], which uses the multi-head attention layer and contains six layers of the encoder and six layers of the decoder.

**Figure 4: Distribution of the MESIA value of the code comments in the TL-CodeSum dataset.**

The third approach is a pre-trained code model named CodeT5 [61], which can be fine-tuned in pairs of code-comment data to generate code comments. These are three representative approaches and other sophisticated approaches [11, 25, 34–36, 57, 60, 62, 67] are more or less based on ideas from these three approaches. Thus, by studying these three approaches, we can get a basic understanding of existing neural approaches.

4.2.3 Experimental Settings. Our experimental environment is a desktop computer equipped with six NVIDIA GeForce GTX 1080Ti GPUs, Intel Xeon CPU, 24GB RAM, running on Ubuntu OS. For the neural approaches, we follow the implementation provided in their respective original papers [9, 28, 61], and adopt the recommended hyperparameter settings [2, 6].

5 STUDY RESULT

In this section, we describe our study results and give some interesting findings.

5.1 RQ1. How well does MESIA reflect the relative extent of supplementary information in method-level comments?

To answer this research question, we first calculate the MESIA value for each code comment in the dataset. After calculation, we find that 99.8% of the MESIA values are less than 10. Therefore, we divide the MESIA values into 11 intervals: [0,1], (1,2], ..., (9,10], and (10,+∞). Then, we analyze how many code comments there are in each interval of the MESIA value. Figure 4 shows the results. We can see that most code comments get a MESIA value between 3 and 6. About 19.3% of code comments get a small-MESIA value that is smaller than 3. About 13.0% of code comments get a large-MESIA value that is larger than 6.

To investigate whether MESIA is consistent with manual assessment of the relative supplementary extent of code comments, we conduct a manual experiment as follows. First, we construct 100 pairs of experimental data by randomly selecting 10 pairs of a

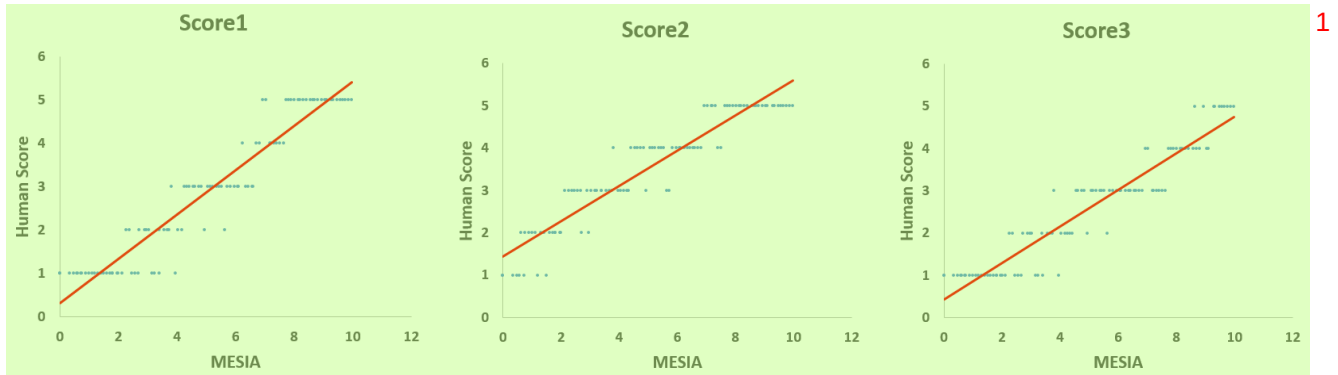


Figure 5: Correlation between the MESIA values and the manual scores of each rater for the experimental code comments. MESIA and Score1 have a Spearman's $\rho=0.9532$ (p-value of $9.84e-53$). MESIA and Score2 have a Spearman's $\rho=0.9530$ (p-value of $1.26e-42$). MESIA and Score3 have a Spearman's $\rho=0.9498$ (p-value of $2.69e-51$). We also calculate the Spearman's ρ between the manual scores of different raters. Score1 and Score2 have a Spearman's $\rho=0.9478$ (p-value of $1.78e-50$). Score1 and Score3 have a Spearman's $\rho=0.9367$ (p-value of $1.77e-46$). Score2 and Score3 have a Spearman's $\rho=0.9780$ (p-value of $1.56e-68$). All the results demonstrate a strong and significant correlation.

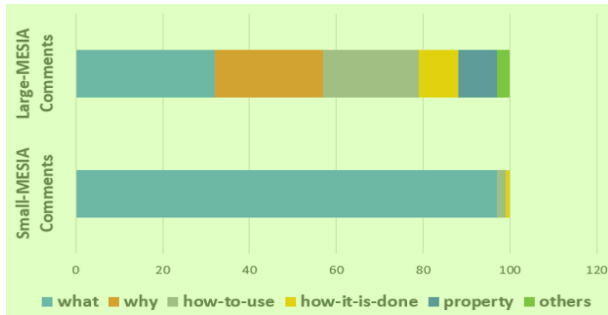


Figure 6: Distribution of different kinds of code comments.

method and its comment from each MESIA value interval². Second, we shuffle these 100 pairs of data and show them to three developers with more than five years of programming experience. We follow the hybrid mode of reading the method-level comment of a method and show only the method signatures and the comment to the developers. These three developers have never seen the data previously. Third, these three developers are asked to rate a score between 1 to 5 for each code comment independently according to their manual perception of the supplementary extent of the code comment. 1 represents that the comment brings little supplementary information and can even be left out. 2 represents that the comment brings a little supplementary information, which can still be derived by digesting the method signature. 3 represents that the comment brings substantive supplementary information to enhance the understanding of the method signature. 4-5 represents that the comment has brought supplementary information beyond

²We merge code comments whose MESIA values are in $(10, +\infty)$ with code comments whose MESIA values are in $(9, 10]$ because the amount of comments in these two intervals is too small.

MESIA	Method and Comment	Category
0.49	Comment: skips from the peek buffer. private int skipFromPeekBuffer (int length) { ... }	What
1.49	Comment: add the specified clipboard listener. public static void addClipboardListener(final ClipboardListener l){ ... }	What
2.29	Comment: handles a change in the current selection. public void handleSelectionChanged(String selection){ ... }	What

Figure 7: Examples of small-MESIA comments.

the method signature. The higher score they rate, the larger extent of supplementary information they perceive the comment to have. Finally, we investigate the correlation between each rater's manual scores and the MESIA values for the experimental comments.

Figure 5 presents the MESIA values and different raters' manual scores (described as Score1, Score2, and Score3, respectively) on the 100 pairs of experimental comments. We can see that although different raters may rate different scores for the same code comment (e.g., the second rater tends to give higher scores), the MESIA values of the comments are highly correlated with the manual scores given by each rater. We also calculate the Spearman's ρ [8] of the MESIA values and the manual scores of each rater. The results indicate that MESIA has a strong and significant correlation with manual assessment of the relative supplementary extent of code comments.

To investigate the difference between small-MESIA comments and large-MESIA comments, we further conduct a manual experiment. We randomly select 100 code comments with a small-MESIA value (<3) and 100 code comments with a large-MESIA value (>6), then ask the three developers to manually classify these code comments into different categories. We follow a clear taxonomy of code comments given by previous work [14]. The taxonomy has

2

MESIA	Method and Comment	Explanation	Category
6.29	Comment: returns null if there is nothing left. public String peek () {...}	This method peeks for the next element and returns null if there is nothing left. The comment describes the supplementary information about the functionality of the method.	What
7.05	Comment: method for BeanContextChild interface. public void firePropertyChange (String name, Object oldValue, Object newValue) {...}	This method is from a class that implements the interface named BeanContextChild. It inherits the corresponding method to accomplish the specification defined in the BeanContextChild interface. The comment explains such design idea.	Why
9.86	Comment: caller must hold rwlock. protected Object[] getAvailableIDsArray () {...}	This method has a critical usage constraints. Every method that tries to call the method must hold the rwlock for synchronization issues. The comment describes such usage constraints.	How-to-use
8.64	Comment: accessed via reflection. public static Boolean isLockGrantor (String serviceName) {...}	This method checks whether the service with the given service name is a lock grantor. The comment explains that the method accomplishes such checking by accessing corresponding information via java reflection.	How-it-is-done
6.70	Comment: java nio replacement of common-io. public void writeStringToFile (Path file, String text) throws IOException {...}	This method uses java nio to write string to file instead of using common io. The comment describes such information, which indicates an essential property because when using java nio, the writing process will never block.	property
7.19	Comment: implementation of visitor pattern. public void accept (AbstractReporter r) {...}	This method implements visitor pattern, which is a well-known design pattern that can add new operations to existing object without modifying it. The comment indicates such external knowledge about the code.	Others

Figure 8: Examples of large-MESIA comments. 1

six categories: (1) “What” describes the functionality of the method. (2) “Why” describes why the method exists. (3) “How-to-use” describes how to use the method, including the method’s usage constraints. (4) “How-it-is-done” describes the implementation details of the method. (5) “Property” describes special properties (e.g., pre-conditions) of the method. (6) “Others” describes other additional external knowledge about the method. Because each code comment in the dataset is only one sentence and tends to describe only a certain aspect of the method, we follow the previous work [14] and ask the developers to classify these code comments into one of the six categories independently. If there is a disagreement between the classifying results, developers will discuss it and come to a consensus. Finally, for the small-MESIA comments, the Fleiss Kappa value [17] of the classifying results is 0.88, which is almost a perfect agreement. For the large-MESIA comments, the Fleiss Kappa value of the classifying results is 0.76, which is also a substantial agreement.

Figure 6 presents the classifying results of the code comments. For the small-MESIA comments, most comments fall into only the “what” comment category (describing the functionality). Figure 7 shows three small-MESIA comments. We can see that the content of these comments can be easily obtained from the method signature. In the hybrid mode of reading the method-level comment to comprehend a method, the comment provides little supplementary information. For the large-MESIA comments, they cover all the six comment categories. Figure 8 shows six large-MESIA comments in different categories. These comments are all real-world examples from the studied dataset, which has been denoised by previous work [50]. After our manual checking, these comments are all relevant to their corresponding methods and accurate (see the explanation). We can see that the large-MESIA comments describe various kinds of essential information about methods. For example,

the comment of the method “getAvailableIDsArray” is “caller must hold rwlock”, which is a vital constraint to use the method. The comment of the method “writeStringToFile” is “java nio replacement of common-io”, which indicates an important property about the method because when using java nio, the writing process will never block. With such code comments, developers can use the methods more safely and with more confidence.

Summary for RQ1 6

MESIA is consistent with manual assessment of the relative supplementary extent of code comments. Small-MESIA comments occupy around 20% of the dataset and mostly fall into only the WHAT comment category while large-MESIA comments can provide various kinds of essential information about methods. 7

5.2 RQ2. What is the capability of existing neural approaches to generate code comments with different MESIA values? 8

As we find in RQ1, the existing dataset is a mixture of code comments with various extents of supplementary information. To answer this research question, we need to consider the MESIA values of the code comments in the test set. 9

We divide the test set into different test groups according to the MESIA values of the comments. The test set contains 5905 pairs of a method and its corresponding comment after denoising and deduplication. We rank these code comments according to their MESIA values and equally divide them into 10 test groups (G1 to G10). The average MESIA values of these 10 test groups are 1.51, 2.70, 3.25, 3.68, 4.07, 4.45, 4.85, 5.27, 5.83, 7.03, respectively. 10

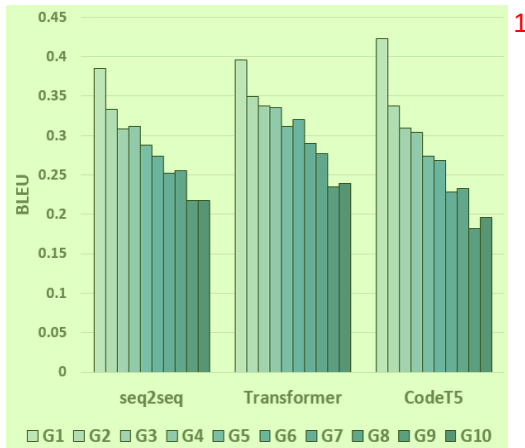


Figure 9: Evaluation results of three models on different MESIA values of test groups.

We train three neural models (seq2seq, Transformer, and CodeT5) on the training set and evaluate their effectiveness on each of the 10 test groups. Figure 9 shows the BLEU scores of the comments generated by the three models in the 10 test groups. We can see that all models tend to achieve higher BLEU scores on test groups with smaller-MESIA comments. As the MESIA value of the reference comments in the test group increases, the effectiveness of all models gradually declines. Take the Transformer model as an example, in test group 1, the BLEU score can reach about 0.4; but in test group 10, the BLEU score is only 0.23.

The results indicate that large-MESIA comments in the dataset are generally more difficult to generate than small-MESIA comments. We suspect that such difficulty may be caused by the following two factors. 1) Large-MESIA comments are more diverse as shown in RQ1. Therefore, the models are difficult to successfully generate the same category of code comments as the reference comments. 2) There are many small-MESIA comments in the training set, which influence the training of the models to pay more attention to the information in the method signature but overlook other essential information needed for generating the large-MESIA comments.

We suggest that the large-MESIA comments shall be given more attention in future research on code comment generation. If the comment-generation approaches fail to generate the small-MESIA comments, developers can still easily obtain the corresponding information from method signature in a hybrid mode of reading the method-level comment to comprehend a method; but if failing to generate the large-MESIA comments, developers may miss essential information needed to comprehend the method.

Summary for RQ2

Based on the current dataset, existing neural approaches tend to get higher BLEU scores in generating the small-MESIA comments. The generation of large-MESIA comments has much room for improvement.

5.3 RQ3. How well can MESIA be used to improve existing neural approaches to generate large-MESIA comments?

As shown in RQ2, the generation of large-MESIA comments has much room for improvement. In this research question, we want to investigate whether we can improve the existing neural approaches to generate large-MESIA comments by improving the training data with the MESIA metric.

We construct three new training sets with the help of the MESIA metric. The training set originally contains 53506 code comments after denoising and deduplication. We rank these code comments according to their MESIA values and equally divide them into 10 groups. Every group contains 5350 pairs of a method and its comment (we omit the last six pairs for convenience). The average MESIA values from group 1 to group 10 are 1.56, 2.71, 3.28, 3.72, 4.12, 4.50, 4.89, 5.32, 5.87, and 7.07, respectively. The distribution is similar to the test groups. Then we construct three new training sets based on these 10 groups of data: the L-training set with groups 1-8 (average MESIA 3.76), the M-training set with groups 2-9 (average MESIA 4.30), and the H-training set with groups 3-10 (average MESIA 4.85). By using these training sets to train the neural models and evaluate the comment-generation result, we can investigate whether refining the training data with the MESIA metric can help existing approaches to generate more large-MESIA comments.

The construction of the three new training sets considers the following two factors. First, they shall contain the same size of sufficient training data. So we choose eight groups from the initial training data to construct the three new training sets and each new training set contains 42800 pairs of a method and its comment. Second, they shall contain a large amount of the same training data (groups 3-8). If the code comments in these training sets are all different, some other factors may also inevitably influence the effectiveness of the existing neural approaches. We need to minimize the influence of other factors.

We retrain the three neural models based on these three new training sets and evaluate their effectiveness again. The test set is also divided into 10 groups as RQ2. Figure 10 presents the BLEU scores of the three models in the 10 test groups. We can see that based on the L-training set, all the approaches tend to achieve low BLEU scores on the large-MESIA test groups. As we gradually adjust the training set from L-training to H-training, the BLEU scores on the large-MESIA test groups gradually improve. The results indicate that different training data do affect the neural models' capability to generate large-MESIA comments. We can improve the generation of large-MESIA comments by reducing the proportion of small-MESIA comments in the training data.

Figure 11 shows the comments generated for the method named "lock". In L-training, the generated comment is an intuitive summary that highly resembles the signature and has a small-MESIA value. In H-training, the generated comment is the same as the reference comment, which has a large-MESIA value and describes how the method accesses corresponding information to accomplish the locking (accessed via reflection). The comment generated under All-training (groups 1-10) is even wrong. The results indicate the usefulness and importance of the MESIA metrics, which can be leveraged to refine the training data to generate comments



Figure 10: Evaluation results of three models on different MESIA values of test groups when using different training sets. 2

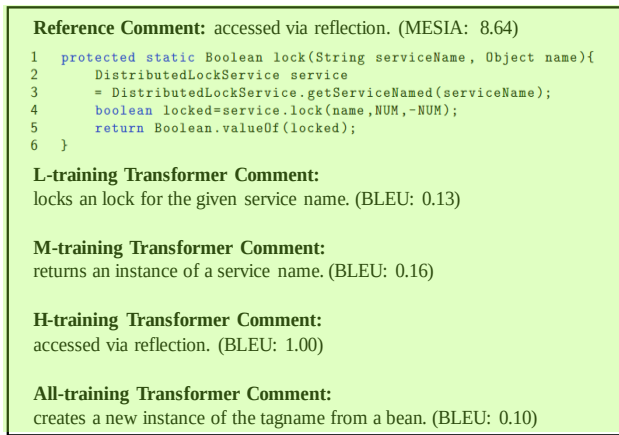


Figure 11: Comments generated for the method named “lock” under different MESIA values of training sets. 4

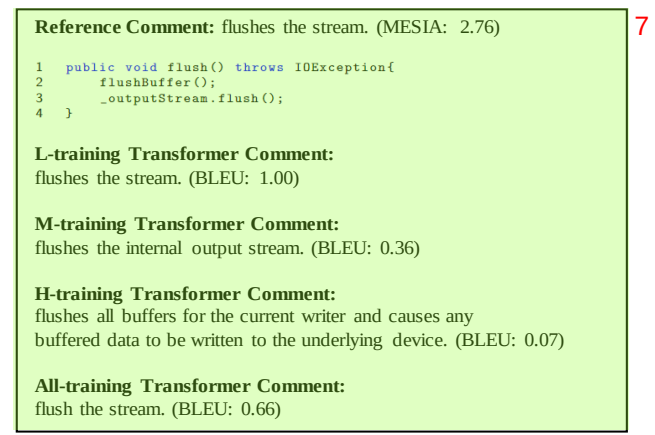


Figure 12: Comments generated for the method named “flush” under different MESIA values of training sets. 8

that bring more supplementary information for developers in a hybrid mode of reading the method-level comment to comprehend a method. 5

For test sets with small-MESIA comments (e.g., G1 and G2), because they contain little supplementary information and H-training models tend to generate comments that bring more supplementary information not presented in the related reference comments, the BLEU score will naturally decline. For example, Figure 12 shows the comments generated for the method named “flush”. In L-training, the generated comment is “flushes the stream”, which is the same as the reference comment and has a perfect BLEU score but just brings a little supplementary information. In H-training, the generated comment is “flushes all buffers for the current writer and causes any buffered data to be written to the underlying device”, which provides more supplementary information. Such a comment has no appropriate ground truth for evaluation and gets a low BLEU score, but is actually pretty good. We search through the Internet and find that the method invokes two APIs in JDK (i.e., *BufferedWriter.flushBuffer()* and *OutputStream.flush()*). According to the JDK documentation [7], the *flushBuffer* API “flushes all buffers for the current writer” and the *flush* API “causes any buffered data to 6

be written to the underlying device”. The documentation confirms the accuracy of the generated comment. The model cannot generate such a comment even if it was trained in All-training, which contains all the data in original training sets. 9

The results reveal the influence of the supplementary nature of method-level comments on automatic comment generation. By reducing the proportion of small-MESIA comments in the training set, the retained model will be more effective in generating large-MESIA comments to better support developers’ hybrid mode of reading the method-level comment to comprehend a method, but will get a lower BLEU score if evaluated in small-MESIA test sets. The results also indicate that with good training data, auto-generated comments can sometimes even surpass human-written reference comments. The latest study [44] on artificial intelligence has also reported that summaries generated by existing popular large language models can surpass the reference summaries in the benchmark dataset. Having no appropriate ground truth for evaluation is an issue. Future work can explore human-like evaluation [18] to better evaluate the automatic generation of code comments. 10

Summary for RQ3 1

We can improve the capability of existing neural approaches to generate large-MESIA comments by reducing the proportion of small-MESIA comments in the training set. The retrained model can generate meaningful large-MESIA comments that surpass the human-written reference comments for methods in the small-MESIA test set, but will get a lower BLEU score in evaluation because of having no appropriate ground truth. 2

6 DISCUSSION 3

In this section, we discuss our work's implications for future work 4 and the threats to validity.

6.1 Discussion of the MESIA Metric 5

6.1.1 Significance of the MESIA metric. Several studies [27, 31] 6 have observed developers' complaints about code comments' lack of supplementary information, and many developers advocate that "comments should not repeat the code" [4] but "describe things that aren't obvious from the code" [42]. Our proposed MESIA metric aims to evaluate the extent of supplementary information in code comments so as to improve the recent increasingly popular neural approaches for generating code comments. In practice, generic comments [27] that do not provide much supplementary information are frequently encountered. Without the MESIA metric, we have no idea about the supplementary nature of code comments in the existing dataset for neural generation of code comments. Using the MESIA metric, we reveal the influence of the supplementary nature of method-level comments on automatic comment generation. We believe that our results can guide the usage of data in future work, and we encourage future work to apply our MESIA metric in more diverse datasets to validate the effectiveness in helping generate comments that better support developers' hybrid mode of reading the method-level comment to comprehend a method.

6.1.2 Challenges of generating large-MESIA comments. There 7 are still many challenges in generating large-MESIA comments. For example, the method named `writeStringToFile` has a large-MESIA reference comment `"java nio replacement of common - io"`, which contains external knowledge about the method. The Transformer model in H-training generates a comment `"writes a string to a file creating the file if it does not exist using the default encoding for the vm."`, which is correct but still cannot provide the supplementary information in the reference comment. Based on our observation, it is common for the large-MESIA comment to contain supplementary information that cannot be directly derived from the associated method. Therefore, instead of just taking the given method as input, future work needs to consider more related information about the method, such as the file context or project context of the method, the documentation of the invoked APIs inside the method body, and other external programming knowledge about the method. In addition, because the large-MESIA comments can be diverse, it is a challenge to determine which kinds of supplementary information a given method may prefer or the developer may intend to write. Future work can explore approaches that can consider developers' intentions [19, 41] to generate comments that better satisfy developers' expectations.

6.1.3 Improvements of the MESIA metrics. Future work can 8 explore improving the MESIA metric in the following aspects. First, the calculation of the MESIA metric involves the probability of each word appearing in code comments. Currently, the probability is tied to the dataset. Therefore, the distribution of the words in the comment dataset may affect the MESIA metric. Future work can calculate the probability via a large scope to alleviate such an issue, or try some other ways of calculation such as the tf-idf. Second, to make the result of the metric comparable across different comment entries, we divide it by the comment length, which may affect the MESIA metric. For some extremely short comments, although the quantity is small in the dataset, the MESIA value may be magnified. Future work can explore some smoothing approaches to alleviate such an issue. Third, although we have stemmed the words in method signature and words in code comments to a normalized form for comparison, future work may also need to further consider the abbreviations and synonyms to calculate the supplementary information amount better. Finally, the MESIA metric currently focuses on the scenario of comprehending and reusing methods in a hybrid mode. Therefore, we consider only the method signature in the metric. In other scenarios, future work needs to consider other additional aspects of the method, such as the invoked-methods' names inside the method body, to better assess the supplementary extent of code comments.

6.2 Threats to Validity 9

The threat to validity in this paper mainly includes the manual 10 assessment process, the exploited dataset, and the exploited neural approaches for generating code comments.

The manual assessment process involves three developers and 11 300 pairs of a method and its comment. Although these three developers all have more than five years of programming experience, their perception of the supplementary extent of code comments may be subjective. Although these 300 pairs of a method and its comment are randomly chosen from different MESIA value intervals and are later shuffled, the amount of code comments may still bring threats to validity. We plan to do the human study on more data with more experienced human participants to further study the MESIA metric.

The exploit dataset contains pairs of a method and its comment 12 constructed by previous work [26, 50], and is further denoised and deduplicated. Although the dataset is widely used by later neural approaches for generating code comments, the results may not generalize to other datasets. We plan to conduct experiments on more datasets to further study the experimental results in the future.

The exploit neural approaches (seq2seq, Transformer, and CodeT5) 13 are all from previous work [9, 28, 61]. Although these are three common types of approaches, more advanced approaches such as large language models remain to be studied. In this paper, we do not adopt the large language models such as ChatGPT [55] because they have learned a large amount of data and the studied dataset (crawled from popular GitHub repositories long before November 2022) has probably been seen by these models and the evaluation will be affected. Nonetheless, our results have indicated that even for the traditional models, as long as trained with good training

data, the auto-generated comments can sometimes surpass human-written reference comments. In the era of large language models, we believe that such a phenomenon will become more prevalent. We leave it as future work to comprehensively study the comment-generation results of large language models.

7 RELATED WORK²

Our work is mainly related to two lines of research: (1) Neural generation of code comments. (2) Code comment assessment. We discuss some of the related work in this section.

7.1 Neural Generation of Code Comments⁴

In recent years, many neural approaches have been proposed for generating code comments [20, 51]. The early work is mainly based on the seq2seq model. The representative work is codenn proposed by Iyer et al. [28], which first introduce the seq2seq model from neural machine translation into code comment generation. Codenn treats the code snippet as a word sequence and translates it to a comment. Besides word sequence, some approaches also use the structural information of code, such as DeepCom proposed by Hu et al. [25], code2seq proposed by Alon et al. [11], ast-attendgru proposed by LeClair et al. [34]. Later, the Transformer model is applied in the comment generation task [9, 38, 57]. For example, Ahmed et al. [9] explore the Transformer model that uses a self-attention mechanism to effectively capture long-range dependencies when generating code comments. After that, some approaches [3, 16, 21, 61] based on pre-training and fine-tuning are becoming popular. Recently, large language models [10, 32, 55] demonstrate remarkable capability in code comment generation.

The preceding work has greatly promoted the automatic generation of code comments. Our work reveals the influence of the supplementary nature of method-level comments on automatic comment generation. By reducing the proportion of small-MESIA comments in the training data, we can generate comments that better support developers' hybrid mode of reading the method-level comment to comprehend a method.

7.2 Code Comment Assessment⁷

How to assess code comments has been discussed in several papers [30, 45, 46, 49, 54]. The assessment approaches can be divided into two categories: manual assessment and automatic assessment.

For manual assessment, existing work mainly asks human participants to manually assess certain aspects of code comments, such as accuracy [39, 52], adequacy [41], and naturalness [28]. Manual assessment is important to ensure the quality of code comments but can be subjective and time-consuming.

For automatic assessment, there are three metrics widely used by existing neural approaches for evaluating the generated code comments: BLEU [43], ROUGE_L [37], and METEOR [12]. Several studies [47, 53] have pointed out that these metrics cannot always reflect the quality of the generated code comments. For example, Stapleton et al. [53] do a human study and show that the BLEU and ROUGE_L do not correlate with program comprehension. Roy et al. [47] reassess these automatic assessment metrics and show that small changes in these metrics cannot guarantee human-perceivable improvement. We suspect that one important

reason is that **these metrics rely on a reference comment to calculate a similarity for the generated comment in evaluation, but overlook the supplementary information of the reference comment.** For example, if the reference comment contains little supplementary information, even a generated comment that achieves the best evaluation result (e.g., BLEU = 1) brings little benefit, while a good supplementary comment will inevitably be undervalued. **Our proposed MESIA metric does not rely on a reference comment. Instead, MESIA aims to evaluate and guarantee the supplementary information for the reference comments. Therefore, MESIA can facilitate existing reference-based metrics for better evaluation.**

Some work [13, 33, 40] has been proposed to understand and leverage the similarity between code and comments. However, the supplementary information of code comments is seldom studied. Within our knowledge, our proposed MESIA metric is the first attempt to assess the supplementary extent of code comments. Assessing such an aspect can better support developers' hybrid mode of reading the method-level comment to comprehend a method.

8 CONCLUSION¹³

In this paper, we have raised the awareness of developers' expectations for the supplementary information of method-level comments when reading the method-level comment for a method in a hybrid mode. We have proposed a new metric named MESIA to evaluate the extent of supplementary information that a code comment can provide beyond the method signature. With the MESIA metric, we have conducted experiments on a popular code-comment dataset and three common types of neural approaches to generate method-level comments. Our experimental results have revealed the influence of the supplementary nature of method-level comments on automatic comment generation. By reducing the proportion of small-MESIA comments in the training set, we can improve the capability of existing neural approaches to generate large-MESIA comments. The generated comments can sometimes even surpass the human-written reference comments, and having no appropriate ground truth for evaluation is an issue that needs to be addressed by future work on automatic comment generation.

In future work, we plan to conduct experiments on more datasets and more neural approaches to further study the MESIA metric. We also plan to further improve neural approaches for generating large-MESIA comments. All of our data including the datasets, the source code, and the evaluation results are available at: <https://github.com/MESIA-CodeComment/MESIA>

ACKNOWLEDGMENTS¹⁶

This work is supported by National Key Research and Development Program of China (Grant No. 2023YFB4503803), National Natural Science Foundation of China under Grant No. 62161146003, and the Tencent Foundation/XPLORER PRIZE.

REFERENCES¹⁸

- [1] 2018. <https://github.com/xing-hu/TL-CodeSum>.
- [2] 2020. <https://github.com/wasiahmad/NeuralCodeSum/tree/master/scripts/java>.
- [3] 2020. <https://huggingface.co/microsoft/CodeGPT-small-java-adaptedGPT2>.
- [4] 2021. <https://stackoverflow.blog/2021/12/23/best-practices-for-writing-code-comments/>.
- [5] 2022. https://github.com/BuiltOnTheRock/FSE22_BuiltOnTheRock.

- [6] 2022. https://github.com/adf1178/PT4Code/blob/main/summarization/finetune_t5_gene.py.
- [7] 2023. <https://docs.oracle.com/en/java/javase/21/docs/api/index.html>.
- [8] 2023. Spearman's rank correlation coefficient. https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient
- [9] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2020. A Transformer-based Approach for Source Code Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL'20)*. 4998–5007.
- [10] Toufique Ahmed and Premkumar Devanbu. 2023. Few-Shot Training LLMs for Project-Specific Code-Summarization. In *Proceedings of the 37th International Conference on Automated Software Engineering (ASE'23)*. 1–5.
- [11] Uri Alon, Shaked Brody, Omer Levy, and Eran Yahav. 2019. code2seq: Generating Sequences from Structured Representations of Code. In *Proceedings of the 7th International Conference on Learning Representations (ICLR'19)*.
- [12] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 65–72.
- [13] Yingkui Cao, Yanzen Zou, and Bing Xie. 2019. Extracting Code-Relevant Description Sentences Based on Structural Similarity. In *Proceedings of the 11th Asia-Pacific Symposium on Internetware (Internetware'19)*. 1–10.
- [14] Qiuyuan Chen, Xin Xia, Han Hu, David Lo, and Shanning Li. 2021. Why My Code Summarization Model Does Not Work: Code Comment Improvement with Category Prediction. *ACM Transactions on Software Engineering and Methodology*. 30, 2 (2021), 1–29.
- [15] Sergio Cozzetti B de Souza, Nicolas Anquetil, and Káthia M de Oliveira. 2005. A Study of the Documentation Essential to Software Maintenance. In *Proceedings of the 23rd Annual International Conference on Design of Communication: Documenting & Designing for Pervasive Information (SIGDOC'05)*. 68–75.
- [16] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CODEBERT: A Pre-Trained Model for Programming and Natural Languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP'20)*. 1536–1547.
- [17] Joseph L. Fleiss. 1971. Measuring Nominal Scale Agreement Among Many Raters. *Psychological Bulletin* 76 (1971), 378–382.
- [18] Mingqi Gao, Jie Ruan, Renliang Sun, Xunjian Yin, Shiping Yang, and Xiaojun Wan. 2023. Human-like Summarization Evaluation with ChatGPT. *arXiv e-prints* (2023), arXiv:2304.02554.
- [19] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2023. Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning. *arXiv e-prints* (2023), arXiv:2304.11384.
- [20] David Gros, Hariharan Sezhiyan, Prem Devanbu, and Zhou Yu. 2020. Code to Comment “Translation”: Data, Metrics, Baseline & Evaluation. In *Proceedings of the 35th International Conference on Automated Software Engineering (ASE'20)*. 746–757.
- [21] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *Proceedings of the 9th International Conference on Learning Representations (ICLR'21)*.
- [22] Hao He. 2019. Understanding Source Code Comments at Large-Scale. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'19)*. 1217–1219.
- [23] Andrew Head, Caitlin Sadowski, Emerson Murphy-Hill, and Andrea Knight. 2018. When Not to Comment: Questions and Tradeoffs with API Documentation for C++ Projects. In *Proceedings of the 40th International Conference on Software Engineering (ICSE'18)*. 643–653.
- [24] Johannes Hofmeister, Janet Siegmund, and Daniel V. Holt. 2017. Shorter Identifier Names Take Longer to Comprehend. In *Proceedings of the 24th International Conference on Software Analysis, Evolution and Reengineering (SANER'17)*. 217–227.
- [25] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep Code Comment Generation. In *Proceedings of the 26th International Conference on Program Comprehension (ICPC'18)*. 200–210.
- [26] Xing Hu, Ge Li, Xin Xia, David Lo, Shuai Lu, and Zhi Jin. 2018. Summarizing Source Code with Transferred API Knowledge. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI'18)*. 2269–2275.
- [27] Xing Hu, Xin Xia, David Lo, Zhiyuan Wan, Qiuyuan Chen, and Thomas Zimmermann. 2022. Practitioners' Expectations on Automated Code Comment Generation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE'22)*. 1693–1705.
- [28] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing Source Code using a Neural Attention Model. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL'16)*. 2073–2083.
- [29] Mira Kajko-Mattsson. 2005. A Survey of Documentation Practice within Corrective Maintenance. *Empirical Software Engineering* 10, 1 (2005), 31–55.
- [30] Ninus Khamis, René Witte, and Juerge Rilling. 2010. Automatic Quality Assessment of Source Code Comments: The JavadocMiner. In *Proceedings of the Natural Language Processing and Information Systems, and 15th International Conference on Applications of Natural Language to Information Systems (NLDB'10)*. 68–79.
- [31] Junaed Younus Khan, Md. Tawkat Islam Khondaker, Gias Uddin, and Anindya Iqbal. 2021. Automatic Detection of Five API Documentation Smells: Practitioners' Perspectives. In *Proceedings of the 28th International Conference on Software Analysis, Evolution and Reengineering (SANER'21)*. 318–329.
- [32] Junaed Younus Khan and Gias Uddin. 2023. Automatic Code Documentation Generation using GPT-3. In *Proceedings of the 37th International Conference on Automated Software Engineering (ASE'23)*. 1–6.
- [33] Dawn J. Lawrie, Henry Feild, and David Binkley. 2006. Leveraged Quality Assessment using Information Retrieval Techniques. In *Proceedings of the 14th International Conference on Program Comprehension (ICPC'06)*. 149–158.
- [34] Alexander LeClair, Siyuan Jiang, and Collin McMillan. 2019. A Neural Model for Generating Natural Language Summaries of Program Subroutines. In *Proceedings of the 41st International Conference on Software Engineering (ICSE'19)*. 795–806.
- [35] Boao Li, Meng Yan, Xin Xia, Xing Hu, Ge Li, and David Lo. 2020. DeepCommenter: A Deep Code Comment Generation Tool with Hybrid Lexical and Syntactical Information. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'20)*. 1571–1575.
- [36] Jia Li, Yongmin Li, Ge Li, Xing Hu, Xin Xia, and Zhi Jin. 2021. EditSum: A Retrieve-and-Edit Framework for Source Code Summarization. In *Proceedings of the 36th International Conference on Automated Software Engineering (ASE'21)*. 155–166.
- [37] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. 74–81.
- [38] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE'21)*. 336–347.
- [39] Paul W McBurney and Collin McMillan. 2014. Automatic Documentation Generation via Source Code Summarization of Method Context. In *Proceedings of the 22nd International Conference on Program Comprehension (ICPC'14)*. 279–290.
- [40] Paul W. McBurney and Collin Mcmillan. 2016. An Empirical Study of the Textual Similarity between Source Code and Source Code Summaries. *Empirical Software Engineering* 21, 1 (2016), 17–42.
- [41] Fangwen Mu, Xiao Chen, Lin Shi, Song Wang, and Qing Wang. 2023. Developer-Intent Driven Code Comment Generation. In *Proceedings of the 45th International Conference on Software Engineering (ICSE'23)*. 768–780.
- [42] John Ousterhout. 2018. Comments Should Describe Things that Aren't Obvious from the Code. In *A Philosophy of Software Design*. Chapter 13, 95–116.
- [43] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL'02)*. 311–318.
- [44] Xiao Pu, Mingqi Gao, and Xiaojun Wan. 2023. Summarization is (Almost) Dead. *arXiv e-prints* (2023), arXiv:2309.09558.
- [45] Pooja Rani. 2021. Speculative Analysis for Quality Assessment of Code Comments. *Proceedings of the 43rd International Conference on Software Engineering: Companion Proceedings (ICSE'21 Companion)*, 299–303.
- [46] Pooja Rani, Arianna Blasi, Natalia Stulova, Sebastiano Panichella, Alessandra Gorla, and Oscar Nierstrasz. 2023. A Decade of Code Comment Quality Assessment: A Systematic Literature Review. *Journal of Systems and Software*. 195, C (2023), 22 pages.
- [47] Devjeet Roy, Sarah Fakhoury, and Venera Arnaoudova. 2021. Reassessing Automatic Evaluation Metrics for Code Summarization Tasks. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'21)*. 1105–1116.
- [48] C. E. Shannon. 1948. A Mathematical Theory of Communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423.
- [49] Ensheng Shi, Yanlin Wang, Lun Du, Junjie Chen, Shi Han, Hongyu Zhang, Dongmei Zhang, and Hongbin Sun. 2022. On the Evaluation of Neural Code Summarization. In *Proceedings of the 44th International Conference on Software Engineering (ICSE'22)*. 1597–1608.
- [50] Lin Shi, Fangwen Mu, Xiao Chen, Song Wang, Junjie Wang, Ye Yang, Ge Li, Xin Xia, and Qing Wang. 2022. Are We Building on the Rock? On the Importance of Data Preprocessing for Code Summarization. In *Proceedings of the 30th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'22)*. 107–119.
- [51] Xiaotao Song, Hailong Sun, Xu Wang, and Jiafei Yan. 2019. A Survey of Automatic Generation of Source Code Comments: Algorithms and Techniques. *IEEE Access*

- 7 (2019), 111411–111428.
- [52] Giriprasad Sridhara, Lori Pollock, and K Vijay-Shanker. 2011. Generating Parameter Comments and Integrating with Method Summaries. In *Proceedings of the 19th International Conference on Program Comprehension (ICPC'11)*. 71–80.
- [53] Sean Stapleton, Yashmeet Gambhir, Alexander LeClair, Zachary Eberhart, Westley Weimer, Kevin Leach, and Yu Huang. 2020. A Human Study of Comprehension and Code Summarization. In *Proceedings of the 28th International Conference on Program Comprehension (ICPC'20)*. 2–13.
- [54] Daniela Steidl, Benjamin Hummel, and Elmar Juergens. 2013. Quality Analysis of Source Code Comments. In *Proceedings of the 21st International Conference on Program Comprehension (ICPC'13)*. 83–92.
- [55] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Qunjun Zhang, Hanwei Qian, Yang Liu, and Zhenyu Chen. 2023. Automatic Code Summarization via ChatGPT: How Far Are We? *arXiv e-prints* (2023), arXiv:2305.12865.
- [56] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to Sequence Learning with Neural Networks. In *Proceedings of the 27th International Conference on Neural Information Processing Systems (NeurIPS'14)*. 3104–3112.
- [57] Ze Tang, Chuanyi Li, Jidong Ge, Xiaoyu Shen, Zheling Zhu, and Bin Luo. 2021. AST-Transformer: Encoding Abstract Syntax Trees Efficiently for Code Summarization. In *Proceedings of the 36th International Conference on Automated Software Engineering (ASE'21)*. 1193–1195.
- [58] T. Tenny. 1988. Program Readability: Procedures Versus Comments. *IEEE Transactions on Software Engineering* 14, 9 (1988), 1271–1279.
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS'17)*. 6000–6010.
- [60] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving Automatic Source Code Summarization via Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Automated Software Engineering (ASE'18)*. 397–407.
- [61] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP'21)*. 8696–8708.
- [62] Bolin Wei, Yongmin Li, Ge Li, Xin Xia, and Zhi Jin. 2021. Retrieve and Refine: Exemplar-based Neural Comment Generation. In *Proceedings of the 35th International Conference on Automated Software Engineering (ASE'21)*. 349–360.
- [63] Scott N Woodfield, Hubert E Dunsmore, and Vincent Yun Shen. 1981. The Effect of Modularization and Comments on Program Comprehension. In *Proceedings of the 5th International Conference on Software Engineering (ICSE'81)*. 215–223.
- [64] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanning Li. 2018. Measuring Program Comprehension: A Large-Scale Field Study with Professionals. *IEEE Transactions on Software Engineering* 44, 10 (2018), 951–976.
- [65] Bai Yang, Zhang Liping, and Zhao Fengrong. 2019. A Survey on Research of Code Comment. In *Proceedings of the 3rd International Conference on Management Engineering, Software Engineering and Service Sciences (ICMSS'19)*. 45–51.
- [66] Juan Zhai, Xiangzhe Xu, Yu Shi, Guanhong Tao, Minxue Pan, Shiqing Ma, Lei Xu, Weifeng Zhang, Lin Tan, and Xiangyu Zhang. 2020. CPC: Automatically Classifying and Propagating Natural Language Comments via Program Analysis. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE'20)*. 1359–1371.
- [67] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2020. Retrieval-based Neural Source Code Summarization. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE'20)*. 1385–1397.